

Aproximação de π pela Série de Gregory-Leibniz: DCA3703 – Programação Paralela

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório apresenta a implementação de um programa em linguagem C para calcular uma aproximação numérica de π utilizando a série de Gregory-Leibniz, dada por:

$$\pi = 4 \sum_{i=0}^{k-1} \frac{(-1)^i}{2i+1} \quad (1)$$

O objetivo é analisar como a acurácia da aproximação evolui conforme o número de iterações k aumenta, relacionando esse ganho de precisão ao custo computacional (tempo de execução) associado.

2 Metodologia

O programa calcula o valor aproximado de π para diferentes valores de k (de 10 a 10^8), medindo o tempo de execução de cada cálculo com a função `clock()` da biblioteca `time.h`. O erro absoluto foi obtido pela diferença entre o valor de referência de π (com 20 casas decimais) e o valor calculado pela série.

3 Resultados

A Tabela 1 apresenta os resultados obtidos.

Tabela 1: Aproximação de π , erro absoluto e tempo de execução por número de iterações

Iterações	π calculado	Erro absoluto	Tempo (s)
10	3.0418396189	$9,98 \times 10^{-2}$	0.000000
100	3.1315929036	$1,00 \times 10^{-2}$	0.000000
1.000	3.1405926538	$1,00 \times 10^{-3}$	0.000000
10.000	3.1414926536	$1,00 \times 10^{-4}$	0.000000
100.000	3.1415826536	$1,00 \times 10^{-5}$	0.001000
1.000.000	3.1415916536	$1,00 \times 10^{-6}$	0.005000
10.000.000	3.1415925536	$1,00 \times 10^{-7}$	0.051000
100.000.000	3.1415926436	$1,00 \times 10^{-8}$	0.522000

4 Análise

4.1 Convergência do erro

Os dados confirmam o comportamento teórico esperado da série de Gregory-Leibniz: o erro absoluto decresce de forma proporcional a $1/k$. A cada aumento de uma ordem de grandeza no número de iterações, o erro diminui aproximadamente pela mesma proporção (uma ordem de grandeza), como pode ser observado diretamente na tabela: passando de $k = 10$ para $k = 10^8$, o erro cai de $\sim 10^{-1}$ para $\sim 10^{-8}$, um ganho de sete ordens de grandeza acompanhando sete ordens de grandeza no número de iterações.

Essa é uma convergência considerada *lenta* quando comparada a outras séries para π (como as baseadas na fórmula de Machin ou no algoritmo de Chudnovsky, que convergem exponencialmente). Por isso, a série de Gregory-Leibniz é didaticamente interessante, mas pouco utilizada na prática para cálculos de alta precisão.

4.2 Custo computacional

O tempo de execução cresce de forma aproximadamente linear com o número de iterações, como esperado de um algoritmo de complexidade $O(k)$: multiplicar k por 10 multiplica o tempo de execução por um fator próximo de 10 (por exemplo, de 10^7 para 10^8 iterações, o tempo passa de 0,051 s para 0,522 s).

4.3 Relação entre precisão e custo

Combinando as duas observações anteriores, chega-se a um ponto central deste experimento: **cada ordem de grandeza adicional de precisão exige uma ordem de grandeza adicional de processamento**. Diferentemente de algoritmos com convergência mais rápida, aqui não há "atalho": obter uma casa decimal a mais de exatidão custa, em média, dez vezes mais tempo de computação. Esse é um exemplo claro de *retornos decrescentes* do ponto de vista prático: o esforço computacional cresce muito mais rápido do que o ganho de informação útil (dígitos corretos) obtido.

5 Reflexão: aplicações reais

Esse padrão — mais processamento gerando ganhos de precisão cada vez mais caros — se repete em diversas áreas da computação científica e da inteligência artificial:

- **Simulações físicas:** em métodos numéricos para equações diferenciais (dinâmica de fluidos, clima, mecânica estrutural), refinar a malha espacial ou reduzir o passo de tempo melhora a precisão do resultado, mas aumenta o custo computacional de forma não linear, frequentemente exigindo o uso de supercomputadores e paralelização massiva.
- **Inteligência artificial:** no treinamento de modelos de aprendizado de máquina, aumentar o número de dados, épocas ou parâmetros do modelo geralmente melhora a acurácia, mas segue uma curva de retornos decrescentes semelhante à observada aqui: dobrar o poder computacional não dobra o desempenho do modelo. Isso motiva a busca por algoritmos e arquiteturas mais eficientes, em vez de depender

apenas de mais poder de processamento (analogamente ao uso de séries de convergência mais rápida no lugar da série de Gregory-Leibniz).

- **Métodos numéricos em geral:** a escolha de um algoritmo com melhor taxa de convergência (como Newton-Raphson em vez de bissecção, ou Chudnovsky em vez de Leibniz) costuma ser mais eficaz do que simplesmente aumentar o número de iterações de um método lento.

6 Conclusão

O experimento confirma que a série de Gregory-Leibniz converge para π de forma lenta e previsível, com erro proporcional a $1/k$ e tempo de execução proporcional a k . Isso evidencia, de forma simples e mensurável, um princípio recorrente em aplicações computacionais que demandam alta precisão: ganhos de exatidão têm custo computacional crescente, e a escolha do algoritmo (não apenas do volume de processamento) é decisiva para viabilizar resultados precisos dentro de restrições práticas de tempo e recursos.

A Código-fonte

O código a seguir foi utilizado para gerar os resultados apresentados neste relatório. A compilação foi feita com o comando `gcc programa.c -o programa -lm`.

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <time.h>
4
5 double gregory_leibniz(int k) {
6     double aux = 0;
7     double sinal = 1.0;
8
9     for (int i = 0; i < k; i++) {
10         aux += sinal / (2*i + 1);
11         sinal = -sinal;
12     }
13
14     return 4 * aux;
15 }
16
17 int main(void) {
18     const double PI_REAL = 3.14159265358979323846;
19
20     int iteracoes[] = {10, 100, 1000, 10000, 100000, 1000000,
21 10000000, 100000000};
22     int n = sizeof(iteracoes) / sizeof(iteracoes[0]);
23
24     printf("%-12s %-15s %-15s %-15s\n", "Iteracoes", "Pi calculado",
25 "Erro absoluto", "Tempo (s)");
26
27     for (int idx = 0; idx < n; idx++) {
28         int k = iteracoes[idx];
29
30         clock_t inicio = clock();
31         double pi = gregory_leibniz(k);
32         clock_t fim = clock();
33
34         double tempo = (double)(fim - inicio) / CLOCKS_PER_SEC;
35         double erro = fabs(PI_REAL - pi);
36
37         printf("%-12d %-15.10f %-15.10f %-15.6f\n", k, pi, erro,
38 tempo);
39     }
40
41     return 0;
42 }
```

Listing 1: Cálculo da aproximação de π pela série de Gregory-Leibniz